

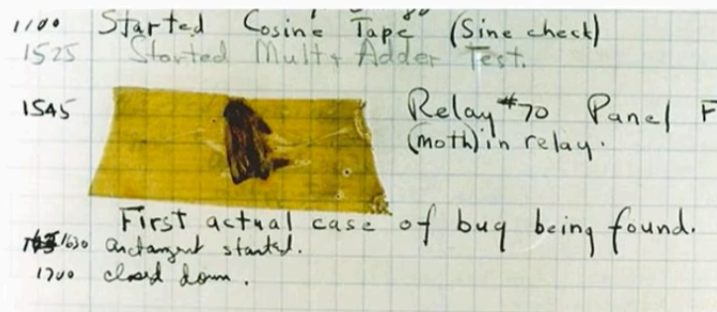
Lecture 28.Exceptions

Handling Errors

Today's Topic: Handling Errors

Sometimes, computer programs behave in non-standard ways

- A function receives an argument value of an improper type
- Some resource (such as a file) is not available
- A network connection is lost in the middle of data transmission



Grace Hopper's Notebook, 1947, Moth found in a Mark II Computer

Exceptions

A built-in mechanism in a programming language to declare and respond to exceptional conditions

Python *raises* an exception whenever an error occurs.

Exceptions can be *handled* by the program, preventing the interpreter from halting.

Unhandled exceptions will cause Python to halt execution and print a *stack trace*.

Mastering exceptions:

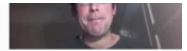
Exceptions are objects! They have classes with constructors.

They enable *non-local* continuations of control:

If **f** calls **g** and **g** calls **h**, exceptions can shift control from **h** to **f** without waiting for **g** to return.

(Exception handling tends to be slow.)

Assert Statements



Assert statements raise an exception of type `AssertionError`

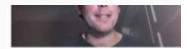
```
assert <expression>, <string>
```

Assertions are designed to be used liberally. They can be ignored to increase efficiency by running Python with the `"-O"` flag. `"O"` stands for optimized.

```
python3 -O
```

Whether assertions are enabled is governed by a bool `__debug__`

Raise Statements



Exceptions are raised with a raise statement.

```
raise <expression>
```

`<expression>` must evaluate to a subclass of `BaseException` or an instance of one.

Exceptions are constructed like any other object. E.g., `TypeError('Bad argument!')`

`TypeError` -- A function was passed the wrong number/type of argument

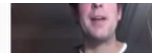
`NameError` -- A name wasn't found

`KeyError` -- A key wasn't found in a dictionary

`RuntimeError` -- Catch-all for troubles during interpretation

Try Statements

Try Statements



Try statements handle exceptions

```
try:
    <try suite>
except <exception class> as <name>:
    <except suite>
...
```

Execution rule:

The `<try suite>` is executed first.

If, during the course of executing the `<try suite>`, an exception is raised that is not handled otherwise, and

If the class of the exception inherits from `<exception class>`, then

The `<except suite>` is executed, with `<name>` bound to the exception.

```
try:
    x = 1/0
except ZeroDivisionError as e:
    print('handling a', type(e))
    x = 0
```

Multiple Try Statements

```
def invert(x):
    y = 1/x
    print('Never inverted if x is 0')
    return y

def invert_safe(x):
    try:
        return invert(x)
    except ZeroDivisionError as e:
        print('handled', e)
        return 0
```

Example: Reduce

```
def reduce(f, s, initial):
    """combine elements of s pairwise using f, starting with initial."""
    if not s:
        return initial
    else:
        first, rest = s[0], s[1:]
        return reduce(f, rest, f(initial, first))

def divide_all(n, ds):
    try:
        return reduce(truediv, ds, n)
    except ZeorDivisionError:
        return float('inf')
```